Schimbare tema, culori, font, etc

- Din meniu “Edit” -> “Preferences”
- De acolo sunt mai multe optiuni
    - Tema: “Colors” -> “Color Schemes”
    - Font: “Font” -> “Font Size”, “Font Name”, “Weight”, etc

Shortcuts

- Pentru simbolul **lambda λ: ctrl + **
- **Reindentare** automata a codului: **ctrl + i**
- **Rulare** a codului: **ctrl + r**

Debugging

- **Tracing**
    - Se importa pachetul: `(require racket/trace)`
    - Se defineste functia si se marcheaza ca traceable: `(trace nume_functie)`
    - La fiecare apel se vor afisa toate apelurile recursive din spate pentru nume_functie **(daca aveti o functie f care apeleaza f-helper si vreti sa vedeti apelurile recursive si pentru f-helper, atunci trebuie sa apelati trace si pe f-helper)**
    - Asta e cea mai simpla varianta, dar daca vreti sa aplicati trace numai pe anumite apeluri sunt mai multe optiuni aici.

```
#lang racket
(require racket/trace)

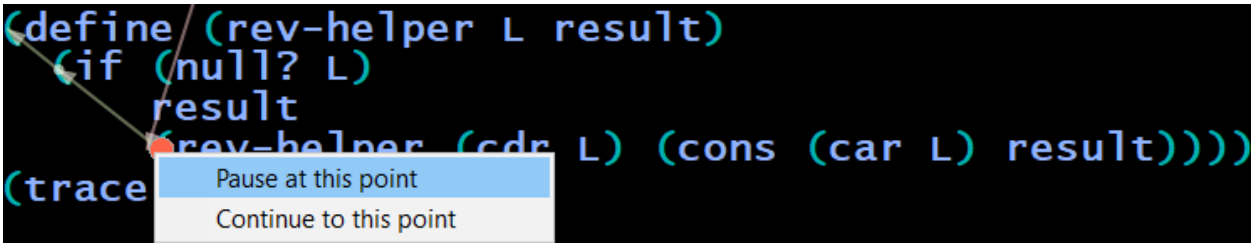
(define (rev-helper L result)
  (if (null? L)
      result
      (rev-helper (cdr L) (cons (car L) result))))
(trace rev-helper)

(define (rev L)
  (rev-helper L '()))
(trace rev)

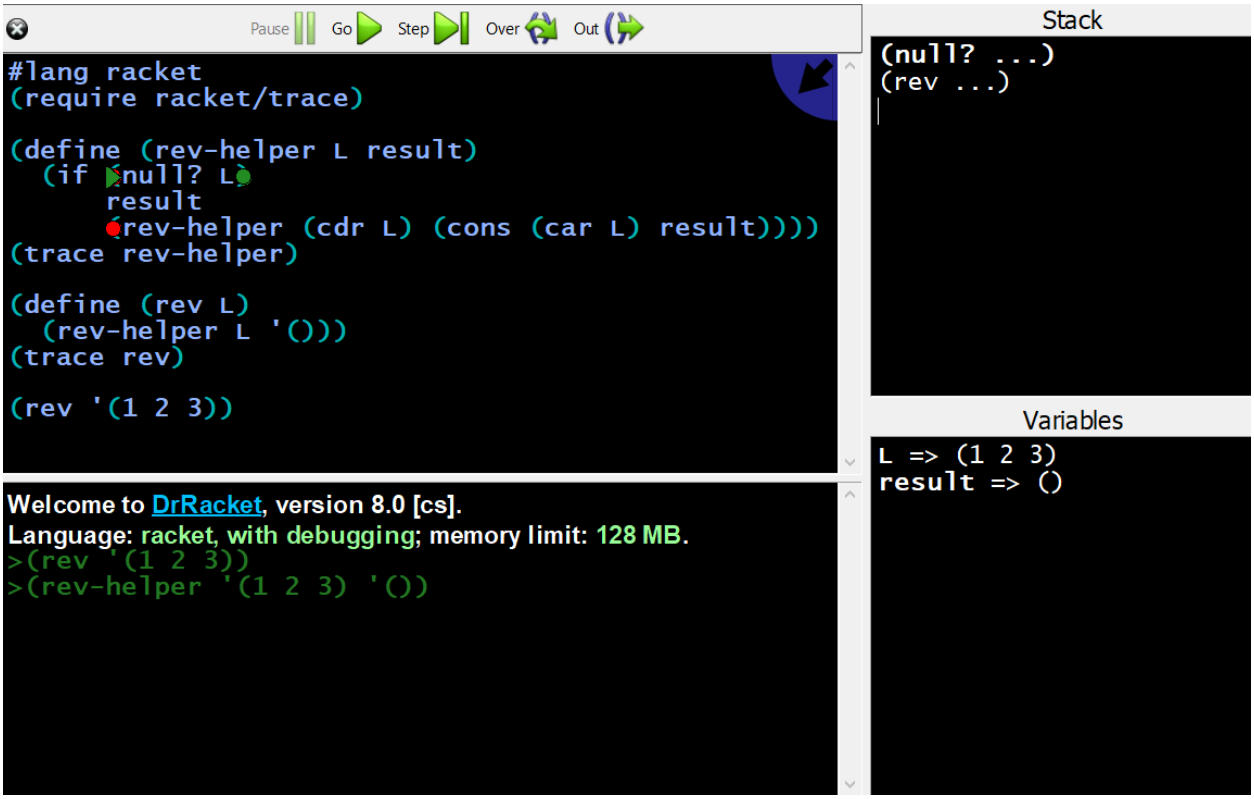
(rev '(1 2 3))
```

```
Welcome to DrRacket, version 8.0 [cs].
Language: racket, with debugging; memory limit: 128 MB.
>(rev '(1 2 3))
>(rev-helper '(1 2 3) '())
>(rev-helper '(2 3) '(1))
>(rev-helper '(3) '(2 1))
>(rev-helper '() '(3 2 1))
<'(3 2 1)
'(3 2 1)
>
```

- **Debug**
    - In dreapta afiseaza variabilele si secventa de apeluri de pe stiva
    - Breakpoints
        - Hover pe ce vreti sa puneti (apel recursiv, conditie, etc) **- paranteze**
        - Ar trebui sa apara o chestie de genul:



        - Pentru breakpoint alegeti pause at this point
    - Apelurile apar in ordine inversa (de la cel mai recent la primul)
    - Variabilele sunt per apel, daca selectati un alt apel de pe stiva se schimba mai jos si variabilele.



- **Check syntax**
    - Afiseaza erorile de sintaxa daca exista
- **Macro stepper**
    - Util pentru debugging macro-uri

Stream-uri

- **Dacă aveți out of memory cel mai probabil un stream ajunge să se evalueze la infinit**
- Când faceți operații pe stream-uri trebuie să aveți un fel de “delay” (ex: stream-cons face cons + delay)
- La un (stream-append s1 s2) dacă s1/s2 sunt numai apeluri recursive o să cicleze pentru că se evaluează la infinit
- O varianta ar fi să scrieți s1/s2 ca (stream-cons x y) pentru că asta pune x și întârzie y
- Sau și mai simplu folosiți (stream-lazy s1) sau (stream-lazy s2) care întârzie evaluarea stream-ului (aceeași idee ca la delay, doar că delay întoarce o promisiune și aici ne trebuie un stream)
- **Dacă aveți eroare cu unbound identifier stream-lazy cel mai probabil aveți Racket v8.0, nu v8.4 (pentru upgrade nu trebuie să dezinstalați nimic, e suficient să instalați v8.4)**
- **Exemplu:**

```
(define (stream-take s n)
  (cond ((zero? n) '())
        ((stream-empty? s) '())
        (else (cons (stream-first s)
                    (stream-take (stream-rest s) (- n 1))))))

; v1: cicleaza pentru ca incearca sa evalueze repeat la infinit
;(define (repeat a b)
;  (stream-append (stream a b) (repeat a b)))

; v2: stream-cons intarzie evaluarea repetitiva a stream-ului
;(define (repeat a b)
;  (stream-cons a (stream-cons b (repeat a b))))

; v3: cea mai simpla cu stream-lazy
; (nu e mereu usor sa se sparga stream-ul cu stream-cons)
(define (repeat a b)
  (stream-append (stream a b) (stream-lazy (repeat a b))))

(stream-take (repeat 'hello 'world) 5)
```